

Analisi di `app.py` — Server di Streaming Video

1. Panoramica Generale

L'applicazione è un **server di streaming video personale** chiamato **KINGSTREAM**, scritto in Python con il framework Flask. Organizza video in categorie/sottocategorie dal filesystem, genera thumbnail automaticamente con FFmpeg e serve i video con supporto allo **streaming parziale (Range Requests)** per consentire lo *scrub* della timeline e la riproduzione efficiente.

2. Architettura del Progetto

```
new_version/
├─ app.py           ← Server Flask (entry point)
├─ templates/
│   ├─ base.html    ← Layout comune (navbar, footer, player overlay)
│   ├─ index.html    ← Homepage con griglia categorie
│   └─ category.html ← Pagina categoria/sottocategoria
├─ static/
│   └─ img/          ← Thumbnail statici (fallback manuali)
├─ stitch/           ← (non analizzato)
└─ .thumbnails/      ← Cache thumbnail generati da FFmpeg (creato runtime)
```

Il file `app.py` (183 righe) gestisce:

Componente	Righe	Descrizione
Config & import	1–13	Librerie, percorsi, costanti
<code>get_categories</code>	20–45	Scansione filesystem → dict nidificato
<code>total_videos</code>	47–53	Conteggio ricorsivo totale video
Route <code>/</code>	55–58	Homepage
Route categoria	60–77	Pagina categoria / sottocategoria
Thumbnail route	79–128	Generazione e cache thumbnail via FFmpeg
Video stream	130–179	Streaming con supporto Range Request

Componente	Righe	Descrizione
Avvio	181–183	<code>app.run(debug=True, port=5001)</code>

3. Analisi Dettagliata del Codice

3.1 Import e Configurazione (righe 1–13)

```
import mimetypes
import subprocess
import shutil
from pathlib import Path
from flask import Flask, render_template, request, Response, abort, send_file

app = Flask(__name__)
BASE = Path(__file__).resolve().parent.parent
VIDEO_FOLDER = BASE / "video"
THUMBNAIL_CACHE = BASE / ".thumbnails"
STATIC_IMG = Path(__file__).resolve().parent / "static" / "img"
FFMPEG_PATH = shutil.which("ffmpeg")
```

Ruolo di ogni libreria:

- `mimetypes` — (importato ma non usato direttamente; potrebbe servire per future espansioni)
- `subprocess` — Lancia FFmpeg come processo esterno per estrarre durata e generare thumbnail
- `shutil` — Usato con `shutil.which("ffmpeg")` per localizzare il binario FFmpeg nel PATH di sistema
- `pathlib.Path` — Gestione moderna e cross-platform dei percorsi filesystem (evita stringhe, usa `/`)
- `Flask` — Micro-framework web: routing, template rendering, request/response
- `send_file` — Invia file statici (thumbnail) direttamente al client

Percorsi:

- `BASE` = `pr_serverfilm/` (due cartelle sopra `new_version/app.py`)
- `VIDEO_FOLDER` = `pr_serverfilm/video/` — cartella con i filmati organizzati in cartelle
- `THUMBNAIL_CACHE` = `pr_serverfilm/.thumbnails/` — cache automatica dei thumbnail
- `STATIC_IMG` = `new_version/static/img/` — thumbnail forniti manualmente
- `FFMPEG_PATH` = percorso di FFmpeg (None se non installato)

3.2 Scansione Filesystem: `get_categories()` (righe 20–45)

```
def get_categories():
    categories = {}
    for entry in sorted(VIDEO_FOLDER.iterdir(), key=lambda e: e.name.lower()):
        if entry.is_dir():
            cat = {"videos": [], "subcategories": {}}
            for f in sorted(entry.iterdir(), key=lambda e: e.name.lower()):
                if f.is_dir():
                    # f è una sottocartella → subcategory
                    sub_videos = sorted(
                        ({ "name": v.name, "path": ..., "description": get_description(v)}
                         for v in sorted(f.iterdir(), ...) if v.suffix == ".mp4"),
                        key=lambda v: v["name"].lower()
                    )
                    cat["subcategories"][f.name] = sub_videos
                elif f.suffix == ".mp4":
                    cat["videos"].append({ "name": f.name, "path": ..., "description": get_description(f.name) })
            categories[entry.name] = cat
        elif entry.suffix == ".mp4":
            categories.setdefault("Generale", {"videos": [], "subcategories": {}})
            categories["Generale"]["videos"].append(...)
    return categories
```

Algoritmo:

1. Scorre ogni elemento in `VIDEO_FOLDER` ordinato alfabeticamente (case-insensitive)
2. Se è una **cartella** → crea una categoria con `videos: []` e `subcategories: {}`
 - Scorre i contenuti della cartella
 - Se trova un'altra **cartella** → è una **sottocategoria**, popolata con i `.mp4` al suo interno
 - Se trova un **file** `.mp4` → lo aggiunge a `videos`
3. Se è un **file** `.mp4` nella root → lo inserisce in una categoria virtuale "Generale"

Struttura dati restituita:

```
{
  "Azione": {
    "videos": [{ "name": "film1.mp4", "path": "Azione/film1.mp4", "description": "..."}],
    "subcategories": {
      "Saga1": [
        { "name": "ep1.mp4", "path": "Azione/Saga1/ep1.mp4", "description": "..."},
        ...
      ]
    }
  }
}
```

```

    },
    "Generale": { "videos": [...], "subcategories": {} }
  }

```

Funzione helper `get_description()` (righe 14–18):

- Cerca un file `.txt` con lo stesso nome del video in `VIDEO_FOLDER/etichetta/`
- Se esiste, ne legge il contenuto come descrizione

3.3 Generazione Thumbnail (righe 79–128)

```

@app.route("/thumbnail/<path:filename>")
def video_thumbnail(filename):
    filepath = VIDEO_FOLDER / filename

```

Algoritmo di risoluzione (priorità decrescente):

1. **Thumbnail manuale in `static/img/`** — Cerca file con stesso nome o stesso nome cartella (`.png` / `.jpg`)
2. **Cache FFmpeg in `.thumbnails/`** — Se già generato in precedenza
3. **Generazione FFmpeg on-demand** — Se FFmpeg è disponibile

Generazione FFmpeg:

```

# 1. Estrae la durata del video
duration_cmd = [FFMPEG_PATH, "-i", str(filepath), "-f", "null", "-"]
result = subprocess.run(duration_cmd, capture_output=True, text=True, timeout=30)

# 2. Calcola seek_time = 20% della durata
seek_time = seconds * 0.2

# 3. Estrae un fotogramma
subprocess.run(
    [FFMPEG_PATH, "-ss", str(seek_time), "-i", str(filepath),
     "-vframes", "1", "-q:v", "5", str(thumb_path)],
    capture_output=True, timeout=30, check=True
)

```

- `"-ss"` — seek alla posizione temporale
- `"-vframes 1"` — estrae un solo fotogramma
- `"-q:v 5"` — qualità JPEG (1-31, minore = migliore)
- Il fotogramma al **20%** del video è spesso rappresentativo (evita intro/credits)

3.4 Streaming Video con Range Request (righe 130–179)

```
@app.route("/video/<path:filename>")
def stream_video(filename):
    filepath = VIDEO_FOLDER / filename
    file_size = filepath.stat().st_size
    range_header = request.headers.get("Range", None)
```

Algoritmo di streaming progressivo (HTTP 206 Partial Content):

1. Il client invia un header `Range: bytes=START-END`
2. Il server apre il file, cerca alla posizione `START` e invia solo i byte richiesti
3. Risposta con status `206` e header `Content-Range`

```
# Parsing dell'header Range
byte1, byte2 = 0, None
range_match = range_header.replace("bytes=", "").split("-")
byte1 = int(range_match[0])
byte2 = int(range_match[1]) if range_match[1] else file_size - 1

# Generatore Lazy: Legge chunk da 8KB
def partial_stream():
    with open(filepath, "rb") as f:
        f.seek(byte1)
        remaining = length
        while remaining > 0:
            chunk_size = min(8192, remaining)
            data = f.read(chunk_size)
            if not data: break
            remaining -= len(data)
            yield data
```

Perché è importante:

- Permette al browser di **cercare** (scrub) in qualsiasi punto del video
- Riduce la banda: l'utente può saltare l'intro senza scaricare l'intero file
- Il lettore video nativo di HTML5 richiede il `206 Partial Content` per funzionare correttamente

Se **non** c'è header `Range`, il file viene inviato per intero (status `200`):

```
def full_stream():
    with open(filepath, "rb") as f:
        while True:
            chunk = f.read(8192)
```

```
if not chunk: break
yield chunk
```

3.5 Routing e Template (righe 55–77)

Route	Template	Descrizione
/	index.html	Homepage con griglia di tutte le categorie
/categoria/<category_name>	category.html	Video/sottocategorie di una categoria
/categoria/<category_name>/<subcategory>	category.html	Video di una specifica sottocategoria

Il template `category.html` è **riutilizzato** per entrambe le viste categoria/sottocategoria:

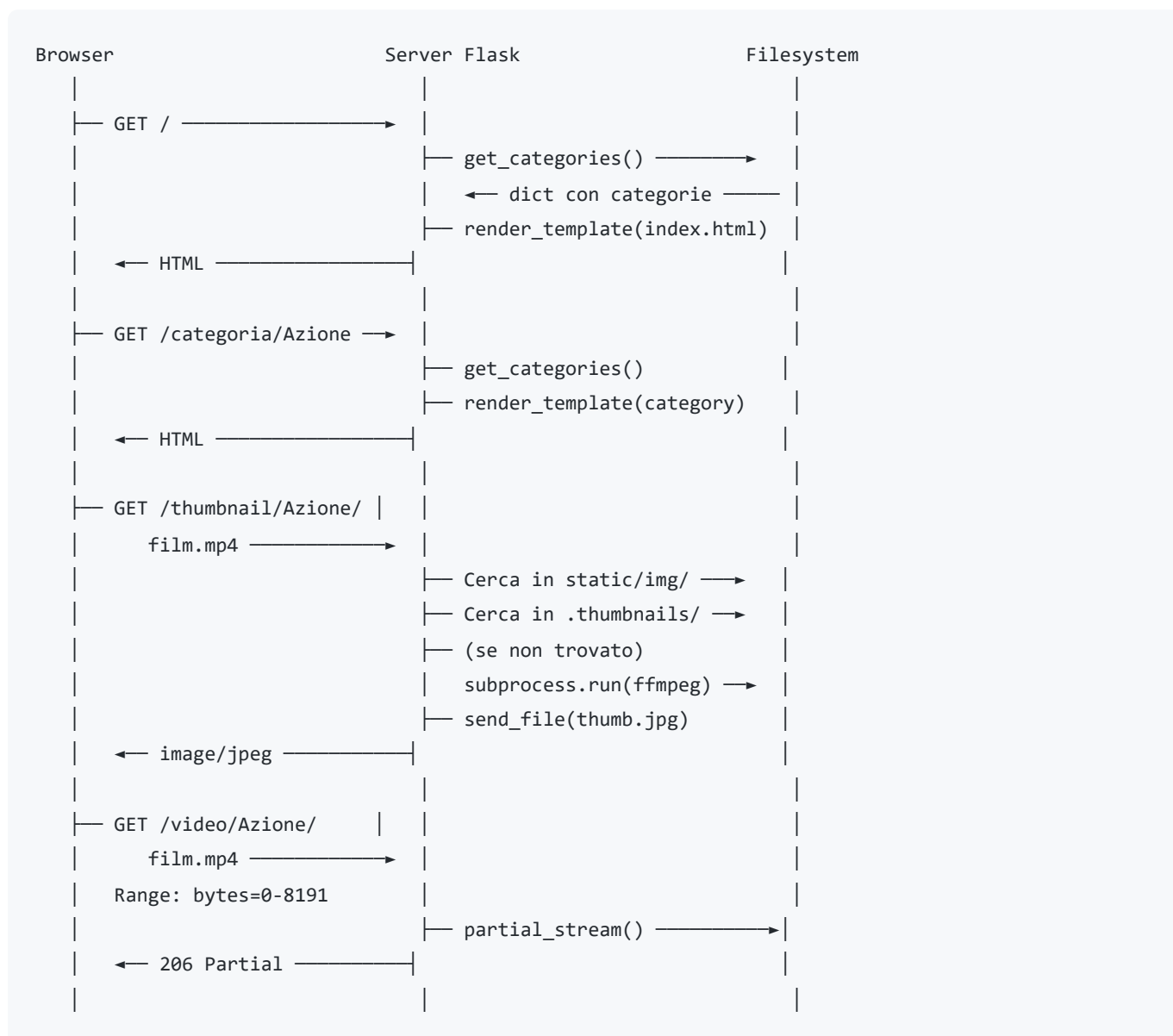
- Se ci sono `subcategories` → mostra una griglia di sottocategorie
- Se ci sono `videos` diretti → mostra la griglia di video con thumbnail

4. Librerie Utilizzate — Ruolo Dettagliato

Libreria	Ruolo in <code>app.py</code>
Flask	Framework web: routing (<code>@app.route</code>), template engine (<code>render_template</code>), gestione richieste/risposte HTTP
pathlib.Path	Gestione moderna dei percorsi: composizione con <code>/</code> , controllo esistenza con <code>.exists()</code> , lettura file con <code>.read_text()</code>
subprocess	Esecuzione di FFmpeg come processo esterno per estrarre durata e generare thumbnail
shutil.which	Ricerca del binario FFmpeg nel PATH di sistema (<code>None</code> se assente)
mimetypes	(importato ma non usato attualmente) — potrebbe servire per dedurre MIME type da estensione
Tailwind CSS	(lato client via CDN) — Framework CSS utility-first per lo styling dell'interfaccia

Libreria	Ruolo in app.py
Google Fonts	(lato client) — Sora, Inter, Geist per la tipografia
Material Symbols	(lato client) — Icone vector-based per l'interfaccia

5. Flusso di Esecuzione Completo



6. Punti di Forza e Possibili Miglioramenti

Punti di Forza

- **Zero dipendenze esterne** oltre Flask e FFmpeg (opzionale)
- **Streaming efficiente** con supporto Range Request (nessun caricamento completo in RAM)
- **Thumbnail automatici** con cache persistente
- **Organizzazione flessibile** dei video: categorie, sottocategorie, descrizioni via file .txt
- **Interfaccia moderna** con Tailwind CSS, modal player overlay, design dark

Possibili Miglioramenti Didattici

- **Cache delle categorie:** `get_categories()` scansiona il disco a ogni richiesta → si potrebbe memorizzare o usare un watch del filesystem
- **Gestione errori FFmpeg:** la generazione thumbnail ha un `except Exception: pass` molto ampio
- **Rate limiting:** proteggere lo streaming da abusi di banda
- **HTTPS:** per `navigator.mediaSession` e riproduzione su dispositivi mobili
- **Test:** aggiungere test unitari con `pytest` e test di integrazione con `Selenium`

7. Confronto: Versione Precedente (`../app.py`) vs Nuova (`new_version/app.py`)

Caratteristica	Versione Precedente	Nuova Versione
Righe di codice	84	183
Sottocategorie	✗ No	✓ Sì (cartelle nidificate)
Descrizioni	✗ No	✓ Sì (file .txt in <code>etichetta/</code>)
Thumbnail	✗ No	✓ Sì (statici + FFmpeg cache)
Percorso video	<code>video/</code> (relativo)	<code>../video/</code> (rispetto a <code>new_version</code>)
Porta	5000	5001
Host	<code>0.0.0.0</code>	<code>127.0.0.1</code> (default Flask)

8. Esecuzione e Requisiti

```
# Requisiti minimi
pip install flask

# Opzionale (per thumbnail automatici)
# Installare FFmpeg e aggiungerlo al PATH

# Avvio
cd new_version
python app.py
# → http://localhost:5001
```

Struttura attesa della cartella `video/` :

```
video/
├─ Azione/
│   ├─ film1.mp4
│   ├─ film2.mp4
│   └─ Saga Rambo/
│       ├─ Rambo 1.mp4
│       ├─ Rambo 2.mp4
│       └─ Rambo 3.mp4
├─ Fantascienza/
│   └─ ...
└─ etichetta/
    ├─ film1.txt          ← descrizione per film1
    ├─ Rambo 1.txt       ← descrizione per Rambo 1
    └─ ...
```